

CSS (Cascading Style Sheets) es un lenguaje que nos permite controlar el aspecto de las páginas web escritas en HTML o en cualquier lenguaje de marcado basado en XML. El lenguaje CSS está creado por el [World Wide Web Consortium \(W3C\)](#), la comunidad internacional que desarrolla estándares que aseguran el crecimiento futuro de la web y vela por conseguir webs disponibles para todo el mundo y desde cualquier dispositivo.

El lenguaje CSS se ha ido creando a lo largo del tiempo en varios niveles. Cada nivel de CSS se ha construido sobre el anterior, generalmente añadiendo funcionalidades nuevas.



1.1. Soporte CSS en los navegadores más utilizados

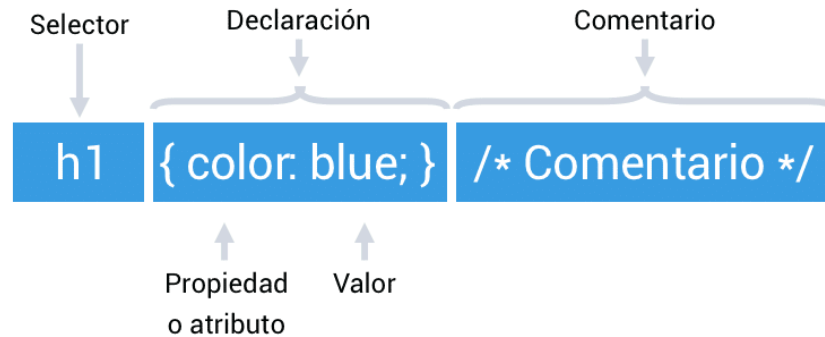
Cada navegador ofrece un soporte CSS distinto. La tabla siguiente detalla el soporte de CSS1, CSS2 y CSS3 de los navegadores más utilizados actualmente.

Navegador	Motor	CSS1	CSS2	CSS3
Chrome	WebKit	Completo	Completo	Selectores, pseudo-clases y ciertas propiedades
Internet Explorer	Trident	Completo	Completo	Selectores, pseudo-clases y ciertas propiedades a partir de la versión 10.0 del navegador
Firefox	Gecko	Completo	Completo	Selectores, pseudo-clases y ciertas propiedades
Safari	WebKit	Completo	Completo	Selectores, pseudo-clases y ciertas propiedades
Opera	Presto	Completo	Completo	Selectores, pseudo-clases y ciertas propiedades

Para la elaboración de la tabla 1.1 se ha tenido como referencia la web [Comparison of layout engines](#), donde se detallan las características CSS de cada navegador.

1.2. Sintaxis CSS

En CSS se utiliza la siguiente sintaxis para asignar valores a las propiedades de cada selector:



- **Selector:** indica sobre qué elemento se aplican los estilos CSS.
- **Propiedad o atributo:** indica qué característica se va a cambiar.
- **Valor:** indica el valor de la propiedad que se desea modificar.
- **Comentario:** los comentarios se escriben entre el carácter de apertura "/*" y el carácter de cierre "*/".

El estándar CSS2 define más de 100 propiedades, cada una de ellas con su lista de valores permitidos. Por su parte, el estándar CSS3 ya incluye más de 200 propiedades o atributos.

2. Estilos CSS en un documento HTML

Hay tres formas de aplicar estilos CSS en un documento HTML, veamos cada una de ellas:

2.1. CSS en etiqueta

Los estilos en línea son declaraciones CSS que se integran en las etiquetas HTML mediante el atributo style. Este método tan solo afecta al elemento en el que se integra el código.

Vamos, por ejemplo, a darle color de fondo a toda la página, para esto le ponemos los estilos a la etiqueta body, que es la que abarca el contenido visible de nuestra página.

```
<body style="background-color: ■ antiquewhite ;">
```

Como podemos ver, se adiciona el atributo style a la etiqueta body, y dentro de este atributo, agregamos la regla background-color con el valor del color que queremos agregar. Ahora agregaremos color de fondo a las secciones de nuestra página.

```
<section style="background-color: ■ aquamarine;">
```

Si quisiéramos adicionar otra regla de estilos a la misma etiqueta, la pondremos separadas por punto y coma ".".

```
<section style="background-color: aquamarine; border: 2px solid blue;">
```

En este ejemplo, adicionamos un borde a la sección, éste borde tiene 2 pixeles de grosor, es un borde hecho de una línea sólida y es de color azul.

Para poder ver todas las posibles reglas aplicables a cada elemento podemos visitar este [enlace](#).

Sin embargo, sólo con la realización de ejercicios y practicando, se irá adquiriendo la habilidad de identificar las reglas que necesitamos usar y las reglas que tenemos disponibles para lograrlo.

No debe utilizar estilos en la etiqueta directamente a menos que realmente tenga que hacerlo! Es realmente malo a la hora de realizar el mantenimiento (puede que tengas que actualizar la misma información varias veces en un mismo documento), y además mezcla tu información CSS para la presentación con tu información HTML para la estructura, lo que dificulta la lectura y la comprensión del código.

2.2. CSS incrustado en la cabecera

Otra manera muy simple de añadir estilo con CSS es utilizando la etiqueta <style> en la cabecera <head> del archivo HTML.

La desventaja de este método es que a la hora de realizar cualquier cambio en un sitio web que contiene múltiples páginas, se debe realizar el cambio en todas las páginas y el código estará repetido.

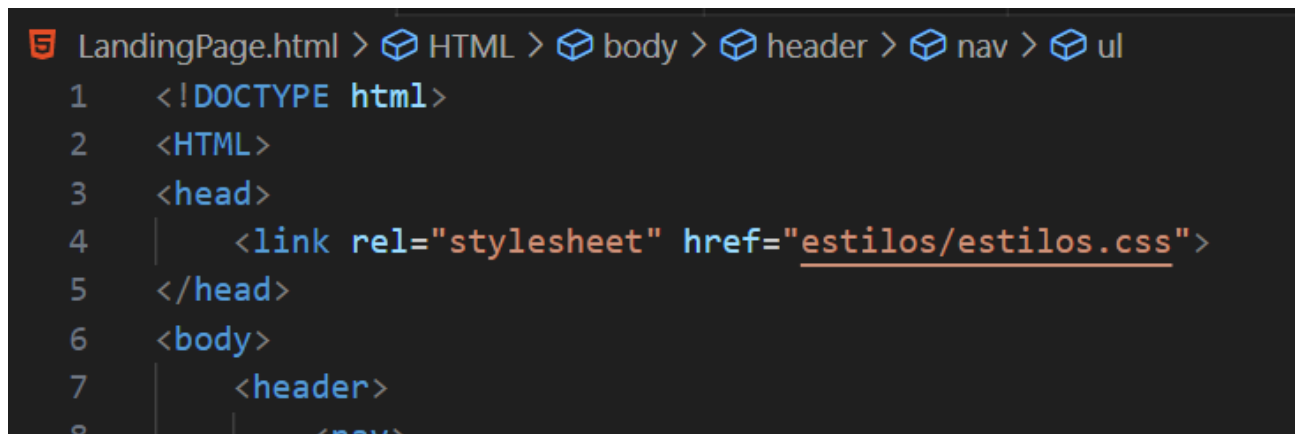
```
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS</title>
  <style>
    body{
      background-color: antiquewhite;
    }
  </style>
</head>
```

Con este método se hace necesario conocer y aplicar los selectores. El concepto de Selector se verá en el siguiente numeral.

2.3. CSS en hojas de estilo externas

Mediante hojas de estilo externas se consigue separar el archivo de estilos del fichero HTML. El archivo de estilos cuenta con la extensión .css y se referencia desde HTML mediante el elemento <link>. Este es el método recomendado para aplicar estilos a una página web.

Este es el método más eficiente y más sencillo de mantener ya que el código CSS se encuentra separado del fichero HTML y en caso de que tengamos un sitio web con cierta cantidad de páginas, solo debemos definir una hoja de estilos y aplicar las mismas reglas a todas las páginas del sitio web, permitiendo la consistencia, mantenibilidad y eficiencia.



Podemos ver la ubicación del archivo css y la etiqueta necesaria para vincularlo a nuestra página web "LandingPage.html"

3. Selectores

3.1. Selector universal

Sintaxis:

```
* {  
  atributo:valor;  
}
```

Ejemplo:

```
* {  
  color: grey;  
} /* El estilo se aplicará a todos los elementos de la página */
```

3.2. Selector etiqueta

Sintaxis:

```
etiqueta { atributo:valor }
```

Ejemplo:

```
p {color: green;} /* El estilo se aplicará a todos los elementos <p> */
```

3.3. Selector clase

Sintaxis:

```
.nombredelaclase { atributo:valor }
```

Ejemplo:

```
.index {color: red;} /* El estilo se aplicará a cualquier elemento que tenga la clase .index*/
```

3.4. Selector identificador (id)

El selector identificador utiliza el atributo id para seleccionar un elemento. Solo puede haber un elemento con un id dado en un documento.

Sintaxis:

```
#nombredelid { atributo:valor }
```

Ejemplo:

```
#cent {color: blue;} /* El estilo se aplicará al elemento que tenga el id #cent */
```

3.5. Selector descendiente

Un elemento es descendiente de otro cuando se encuentra entre las etiquetas de apertura y de cierre del elemento padre. Su sintaxis es: selector1 selector2... selectorN. Siendo el selector N el elemento sobre el que se aplica el estilo.

```
selector1 selector2 selectorN{  
    propiedad: valor;  
}
```

3.6. Combinación de selectores

La combinación de selectores nos permite dar un estilo a todos los selectores indicados.

```
selector1, selector2, selector3{  
    propiedad: valor;  
}
```

3.7. Selector de hijos

Se usa para seleccionar un elemento que es hijo de otro elemento y se indica mediante el signo "mayor que" (>).

```
selector1 > selector2{  
    propiedad: valor;  
}
```

3.8. Selector adyacente

Se usa para seleccionar elementos que son hermanos, es decir, su elemento padre es el mismo y están seguidos en el código HTML. El elemento hermano debe ESTAR INMEDIATAMENTE DESPUÉS. Se indica mediante el signo "más" (+).

```
selector1 + selector2{
```

```
    propiedad: valor;  
}
```

Ejemplos

```
p {  
  background-color: grey  
} /* Selector etiqueta */  
  
.clase {  
  color: red  
} /* Selector clase */  
  
#ident {  
  color: green  
} /* Selector identificador */  
  
* {  
  font-style: italic  
} /* Selector universal */  
  
p a {  
  background-color: orange  
} /* Selector descendiente */
```

[LISTA DE SELECTORES](#) (Revisar los selectores de atributos)

Al aplicar estilos simultáneamente sobre el mismo elemento tener en cuenta:

1. **Origen y relevancia:** ¿De dónde viene la regla? (estilos del usuario, del autor, de user-agent). Las reglas **!important** ganan sobre todo (pero son una mala práctica).
2. **Especificidad:** ¿Qué tan específico es el selector? (lo veremos abajo).
3. **Orden de aparición:** Si todo lo demás es igual, gana la última regla declarada.

Ejemplo argumentado:

CSS



```
1  /* Regla 1 */  
2  p { color: red; }  
3  
4  /* Regla 2 - Gana porque aparece después */  
5  p { color: blue; }
```

La **herencia** significa que ciertos elementos reciben automáticamente los estilos de sus elementos padres. No todas las propiedades se heredan.

Se heredan: propiedades tipográficas (`color`, `font-family`, `line-height`, `text-align`).

NO se heredan: propiedades de caja (`margin`, `padding`, `border`, `background`).

Ejemplo:

html



```
1 <body>
2   <article>
3     <h1>Título</h1>
4     <p>Párrafo con <strong>texto fuerte</strong></p>
5   </article>
6 </body>
```

CSS



```
1 body {
2   color: #333; /* Se hereda a article, h1, p, strong */
3   font-family: sans-serif; /* También se hereda */
4   background: white; /* NO se hereda */
5 }
```

Argumento pedagógico: La herencia es lo que hace eficiente a CSS. Si no existiera, tendríamos que declarar `color` en cada elemento.

Especificidad

X O O O O	! IMPORTANT
X O O O	ESTILOS EN LÍNEA
X O O	#id
X O	CLASES, ATRIBUTOS Y PSEUDOCASES.
X	ELEMENTOS Y PSEUDOELEMENTOS
O	SELECTOR UNIVERSAL

4. Unidades de medida

Los valores de las propiedades se pueden expresar en longitudes absolutas y longitudes relativas. Veamos las diferentes opciones en la siguiente tabla (**Actualizada en 2026**).

## UNIDADES ABSOLUTAS			
<code>px</code>	Absoluta	Píxeles. Unidad basada en la resolución de la pantalla. 1px = 1/96 de 1in. Es fija y no se escala con el tamaño de fuente del usuario.	<code>width: 300px;</code> <code>border: 1px solid black;</code> Ideal para: Bordes finos, sombras, cuando necesitas precisión exacta.
<code>cm</code>	Absoluta	Centímetros. 1cm = 96px/2.54 ≈ 37.8px. Basado en medidas físicas reales.	<code>width: 10cm;</code> Ideal para: Estilos de impresión (print media queries).
<code>mm</code>	Absoluta	Milímetros. 1mm = 1/10 de 1cm.	<code>margin: 5mm;</code> Ideal para: Documentos para imprimir con medidas precisas.
<code>in</code>	Absoluta	Pulgadas (inches). 1in = 2.54cm = 96px.	<code>width: 8.5in;</code> Ideal para: Tamaño de papel en impresión (carta, A4).
<code>pt</code>	Absoluta	Puntos (points). 1pt = 1/72 de 1in ≈ 1.33px. Tradicional en tipografía impresa.	<code>font-size: 12pt;</code> Ideal para: Documentos PDF o impresión donde se requiere tipografía tradicional.
<code>pc</code>	Absoluta	Picas. 1pc = 12pt ≈ 16px. Unidad tipográfica antigua.	<code>margin: 2pc;</code> Ideal para: Diseño editorial impreso (poco usado en web).

Cuándo usar: Bordes finos, sombras, cuando necesitas precisión exacta que no debe escalar.

Otras unidades absolutas

- `cm`, `mm`, `in`: Para impresión
- `pt`: Tipografía impresa (1pt = 1/72 pulgada)
- `pc`: Picas (1pc = 12pt)

Argumento: En web moderna, prácticamente solo usamos `px` de las absolutas. Las demás son para impresión.

## UNIDADES RELATIVAS AL FONT- SIZE			
em	Relativa	Relativo al <code>font-size</code> del elemento padre . Si el padre tiene 16px, 1em = 16px. Se puede compounding (acumular).	<code>padding: 1.5em;</code> <code>font-size: 1.2em;</code> Ideal para: Espaciado interno que escale con el texto, componentes reutilizables.
rem	Relativa	"Root em". Relativo al <code>font-size</code> del elemento raíz (<code><html></code>). Por defecto 1rem = 16px. No tiene compounding.	<code>font-size: 1rem;</code> <code>margin: 2rem;</code> <code>padding: 1.5rem;</code> Ideal para: Tipografía, márgenes, paddings globales. La unidad más recomendada para web moderna.
ex	Relativa	Altura de la letra "x" minúscula en la fuente actual. Aproximadamente la mitad de 1em.	<code>line-height: 2ex;</code> Ideal para: Ajustes tipográficos muy específicos (poco usado).
ch	Relativa	Ancho del carácter "0" (cero) en la fuente actual. Aproximadamente el ancho de un carácter.	<code>width: 60ch;</code> <code>max-width: 80ch;</code> Ideal para: Longitud óptima de lectura (60-80ch es ideal para párrafos).

buscar sobre recomendaciones y posibles problemas en estas unidades de medida

## UNIDADES RELATIVAS AL VIEWPORT			
<code>vw</code>	Viewport	1vw = 1% del ancho del viewport (ventana del navegador). 100vw = ancho total.	<code>width: 50vw;</code> <code>font-size: 4vw;</code> Ideal para: Layouts fluidos, tipografía que escala con el ancho de pantalla.
<code>vh</code>	Viewport	1vh = 1% de la altura del viewport. 100vh = altura total.	<code>height: 100vh;</code> <code>min-height: 50vh;</code> Ideal para: Secciones que ocupan toda la pantalla (hero sections), footers fijos al bottom.
<code>vmin</code>	Viewport	1vmin = 1% del valor más pequeño entre vw y vh.	<code>width: 50vmin;</code> <code>height: 50vmin;</code> Ideal para: Elementos cuadrados que deben caber siempre en pantalla (logos, avatares).
<code>vmax</code>	Viewport	1vmax = 1% del valor más grande entre vw y vh.	<code>width: 50vmax;</code> Ideal para: Elementos decorativos de fondo que deben cubrir siempre.
<code>svw / svh</code>	Viewport	"Small viewport". Tamaño del viewport cuando la UI del navegador está visible (móvil con barra).	<code>height: 100svh;</code> Ideal para: Altura completa en móviles considerando barras del navegador.
<code>lvw / lvh</code>	Viewport	"Large viewport". Tamaño del viewport cuando la UI del navegador está colapsada/oculta.	<code>height: 100lvh;</code> Ideal para: Cuando quieres maximizar el espacio en móviles.

Normalmente es recomendable usar unidades relativas en la medida de lo posible, ya que mejora la accesibilidad de la página web y permite que los documentos se adapten fácilmente a cualquier medio. En concreto, el organismo W3C, recomienda el uso de la unidad em para indicar el tamaño del texto.

4.1. Utilización de píxeles:

El único inconveniente que puede darnos trabajar con píxeles es que no todas las pantallas son iguales y que un píxel no es igual en todas las pantallas. Este inconveniente se soluciona fácilmente siempre que se pueda redimensionar el texto.

4.2. Utilización de ems:

El tamaño de los ems se establece en base al tamaño que tenga definido el navegador. El único problema es que el usuario haya modificado ese tamaño base, aunque los navegadores permiten ajustar estos parámetros nuevamente.

4.3. Convertidor píxel a Ems

Normalmente el tamaño de una fuente por defecto en los navegadores es de 16px (16px = 1em). Por lo que tendríamos la siguiente conversión:

px	em	%
12	0,7500	75
14	0,8750	87,50
16	1,0000	100,00
18	1,1250	112,50
20	1,2500	125

Leer en detalle información sobre [valores y unidades en CSS \(gráficos explicativos de las unidades\)](#)

5. Colores y fondo

Los valores de los colores se pueden definir mediante su nombre, en hexadecimal o mediante los valores RGB (Red, Green, Blue) , HSL (Hue, Saturation, Lightness), RGBA (Red, Green, Blue, Alpha) y HSLA (Hue, Saturation, Lightness, Alpha).

5.1. Tabla de colores básicos

HTML soporta alrededor de 140 nombres de colores diferentes. Entre ellos puedes encontrar los siguientes colores básicos:

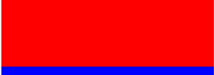
Color	Nombre	HEX	RGB
	black	#000000	0,0,0
	white	#ffffff	255,255,255
	red	#ff0000	255,0,0
	blue	#0000ff	0,0,255
	yellow	#ffff00	255,255,0
	gray	#808080	128,128,128
	green	#008000	0,128,0

Table 5.1. Colores básicos

5.2. Propiedades más utilizadas

Algunas de las propiedades relacionadas con el color y el fondo más utilizadas son las siguientes:

Propiedad	Descripción	Valores
<code>color</code>	Color del texto	RGB HSL HEX nombre del color RGBA HSLA
<code>background-color</code>	Color de fondo	url(...) none
<code>background-image</code>	Imagen de fondo	none underline overline line-through *
<code>background-repeat</code>	Repetición de la imagen de fondo	repeat repeat-x repeat-y no-repeat
<code>background-attachment</code>	Desplazamiento de la imagen de fondo	scroll fixed
<code>background-position</code>	Posición de la imagen de fondo	percentage length left center right
<code>Opacity</code>	Transparencia de un elemento	[0 – 1]

Tabla 5.2: Propiedades colores y fondo

Ejemplo:

```
h1 { background-color: red; }  
h2 { background-color: #FF0000; }  
h3 { background-color: RGB(255,0,0); }  
p { background-color: HSL(0,100%,50%) }  
body { background-image: url("https://bit.ly/2slA7jo"); }
```

En este punto del documento, usted deberá construir un ejemplo donde pueda probar al menos 4 de las propiedades descritas de la tabla, a cada propiedad asigne valores diferentes para que observe los cambios. Como ejemplo, podría crear una página con 3 bloques (div) de una altura de 400px y un ancho de 100%. En el primer bloque escriba un párrafo y asigne un color al texto y otro al fondo del bloque. En el segundo bloque asigne una imagen de fondo en mosaico. Al tercer bloque agregue una imagen de fondo a la derecha y semitransparente. Por último crea un bloque (el cuarto), con un ancho de 100% y una altura de 1500px. Utilice una imagen de fondo y aplique un desplazamiento (scroll).

6. Propiedades de texto

Las propiedades de texto son las que nos permiten controlar el texto como bloque, es decir, afectan al interlineado, a la separación entre palabras, al tabulado, etc. En la tabla 6.1 se muestran las propiedades de texto más utilizadas.

Propiedad	Descripción	Valores
<code>text-indent</code>	Desplazamiento de la primera línea del texto	longitud porcentaje
<code>text-align</code>	Alineamiento del texto	left right center justify
<code>text-decoration</code>	Efectos de subrayado, tachado	none underline overline line-through *
<code>letter-spacing</code>	Espacio entre caracteres	normal longitud
<code>word-spacing</code>	Espacio entre palabras	normal longitud
<code>text-transform</code>	Transformación a mayúsculas / minúsculas	capitalize uppercase lowercase none
<code>line-height</code>	Tamaño del espacio entre líneas	longitud porcentaje
<code>vertical-align</code>	Alineación vertical	top middle bottom

Tabla 6.1: Propiedades de texto

Para estudiar las propiedades de texto, abra el folder que contiene el código de ejemplo y diríjase a las carpetas descritas a continuación:

- Text-indent:
Abra el archivo ejemplo.html y el archivo css. Al visualizar el ejemplo en el navegador, revise la información que se muestra, ¿entiende todas las unidades de medida?, cambie algunos valores del archivo css y observe los cambios en el navegador. ¿Qué sucede al disminuir el tamaño de la ventana? ¿Cuáles unidades de medida se afectan? **Si considera necesario, agregue comentarios en el código para una lectura posterior.**

- Text-align:
Abra el archivo ejemplo.html y el archivo css. Al visualizar el ejemplo en el navegador, revise la información que se muestra, ¿entiende los valores dados a cada clase?, cambie algunos valores del archivo css y observe los cambios en el navegador. ¿Qué sucede al disminuir el tamaño de la ventana? ¿Cuáles unidades de medida se afectan? Si considera necesario, agregue comentarios en el código para una lectura posterior.

- Text-decoration:
Abra el archivo ejemplo.html y el archivo css. Al visualizar el ejemplo en el navegador, revise la información que se muestra, ¿entiende los valores dados a cada clase? ¿Puedo poner varios en una sola regla?, cambie algunos valores del archivo css y observe los cambios en el navegador. ¿Qué sucede al disminuir el tamaño de la ventana? ¿Cuáles unidades de medida se afectan? Si considera necesario, agregue comentarios en el código para una lectura posterior.

6.4. Letter-spacing

```
.a { letter-spacing: 3px; }  
.b { letter-spacing: 10px; }  
.c { letter-spacing: 15px; }
```

6.5. Word-spacing

```
.a { word-spacing: 5px; }  
.b { word-spacing: 20px; }  
.c { word-spacing: 50px; }
```

6.6. Text-transform

```
.a { text-transform: capitalize; }  
.b { text-transform: uppercase; }  
.c { text-transform: lowercase; }
```

6.7. Line-height

```
.a { line-height: 50px; }  
.b { line-height: 3em; }  
.c { line-height: 30%; }
```

Utilice los numerales 6.4, 6.5, 6.6 y 6.7 en el código de ejemplo de text-align, uno a uno analice los cambios que se producen en la página. Agregue comentarios al código para su posterior estudio.

```
.a { text-align: center;
    letter-spacing: 3px;
    word-spacing: 5px;
    text-transform: capitalize;
}
```

7. Propiedades de las fuentes

Las propiedades CSS de las fuentes son las que permiten controlar el tamaño, el tipo, el grosor o el estilo de las letras entre otras cosas.

Propiedad	Descripción	Valores
<code>font-family</code>	Familias de fuentes	nombre-familia nombre-familia-genérica *
<code>font-style</code>	Estilo de la fuente	normal italic oblique
<code>font-variant</code>	Convierte a mayúsculas manteniendo un tamaño inferior	normal small-caps
<code>font-weight</code>	Anchura de los caracteres. Normal = 400, Negrita = 700	normal bold bolder lighter 100 200 300 400 500 600 700 800 900
<code>font-size</code>	Tamaño de la fuente	xx-small x-small small medium large x-large xx-large larger smaller longitud porcentaje

Tabla 7.1: Propiedades de las fuentes

<https://fonts.google.com>

En el código de ejemplo de la carpeta Font, abra el código de ejemplo.html. Analice la propiedad font-family y a continuación pruebe las siguientes reglas:

7.2. Font-style

```
.a { font-style: normal; }
.b { font-style: italic; }
.c { font-style: oblique; }
```

7.3. Font-variant

```
.a { font-variant: normal; }
```

```
.b { font-variant: small-caps; }
```

7.4. Font-weight

```
.a { font-weight: normal; }  
.b { font-weight: bold; }  
.c { font-weight: 400; }
```

7.5. Font-size

```
.a { font-size: 20px; }  
.b { font-size: 2em; }  
.c { font-size: xx-small; }  
.d { font-size: small; }  
.e { font-size: 250%; }
```

8. Propiedades de las listas

Las propiedades CSS de las listas son las que nos permiten controlar los estilos de los marcadores y la posición de los elementos dentro de las listas. En la tabla 8.1 se muestran las propiedades de las listas más utilizadas.

Propiedad	Descripción	Valores
<code>list-style-type</code>	Estilo aplicable a los marcadores visuales de las listas	<code>disc</code> <code>circle</code> <code>square</code> <code>decimal</code> <code>decimal-leading-zero</code> <code>lower-roman</code> <code>upper-roman</code> <code>lower-greek</code> <code>lower-latin</code> <code>upper-latin</code> <code>armenian</code> <code>georgian</code> <code>lower-alpha</code> <code>upper-alpha</code> <code>none</code>
<code>list-style-image</code>	Imagen aplicable a los elementos de las listas	<code>url("http://...")</code> <code>none</code>
<code>list-style-position</code>	Posición del marcador dentro de la lista	<code>inside</code> <code>outside</code>
<code>list-style</code>	Permite establecer el estilo de la lista, la imagen y/o la posición	<code>list-style-type</code> <code>list-style-position</code> <code>list-style-image</code>

Tabla 8.1: Propiedades de las listas

En el código de ejemplo de la carpeta Listas, abra el código de ejemplo.html.

Analice la propiedad `font-family` y a continuación pruebe las siguientes reglas (agregue comentarios para su posterior estudio):

8.1. List-style-type

```
.a {list-style-type: circle;}  
.b {list-style-type: square;}  
.c {list-style-type: upper-roman;}  
.d {list-style-type: lower-alpha;}
```

8.2. List-style-image

```
ul.a { list-style-image: url('https://i.racionalista.pl/img/em/lol.gif'); }  
ul.b { list-style-image: url('https://media.giphy.com/media/tBik70kZOKOk/giphy.gif'); }
```

8.3. List-style-position

```
ul.a { list-style-position: outside; }  
ul.b { list-style-position: inside; }
```

8.4. List-style

```
ul.a { list-style: square inside; }  
ul.b { list-style: circle inside; }
```

8.5. Lista convertida en barra de navegación

Gracias a los estilos podemos convertir una lista en una botonera horizontal, en una galería de fotos u en otros elementos. En el siguiente ejemplo puedes ver una barra de navegación bastante típica que puedes encontrar en cualquier web.

Presta atención a que no se han utilizado clases, sólo se ha necesitado hacer uso de selectores etiqueta y selectores descendientes. Además fíjate que gracias al padding que se le ha incluido en los enlaces, serían perfectamente pulsables.

Ver ejemplo Barra de navegación del material de ejemplo. Realice cambios en las fuentes, los colores de fondo y del texto, luego realice cambios en el padding, intente agregar una imagen como icono en la barra de navegación.

9. Propiedades de las tablas

Las propiedades CSS de las tablas son las que nos permiten controlar los estilos de los títulos de la tabla, el tamaño de las celdas, las filas y las columnas o espaciado entre los bordes. En la tabla 9.1 se muestran las propiedades de las tablas más utilizadas.

Propiedad	Descripción	Valores
<code>caption-side</code>	Posición del título respecto la tabla	top bottom
<code>table-layout</code>	Formato de las celdas, filas y columnas	auto fixed
<code>border-collapse</code>	Selección del modelo de los bordes	collapse separate
<code>border-spacing</code>	Espaciado entre los bordes de celdas adyacentes	longitud
<code>empty-cells</code>	Visibilidad de los bordes de celdas sin contenido	show hide

Tabla 9.1: Propiedades de las tablas

Ver ejemplo Tablas del código de ejemplo. Aplique los estilos de cada numeral a continuación, uno por uno. Adicione comentarios en cada uno para su posterior estudio. en el numeral table-layout, ¿Qué función realiza el valor “auto”?

9.1. Caption-side

```
.a { caption-side: bottom; }  
.b { caption-side: top; }
```

9.2. Table-layout

```
.a { table-layout: auto; width: 200px; }.  
b { table-layout: auto; width: 100%; }.  
c { table-layout: fixed; width: 200px; }
```

9.3. Border-collapse

```
.a { border-collapse: separate; }  
.b { border-collapse: collapse; }
```

9.4. Border-spacing

```
.a { border-collapse: separate; border-spacing: 50px 10px; }  
.b { border-collapse: collapse; border-spacing: 40px; } /* Solo tiene efecto cuando border-collapse: separate; */
```

9.5. Empty-cells

```
.a { empty-cells: hide; }  
.b { empty-cells: show; }
```

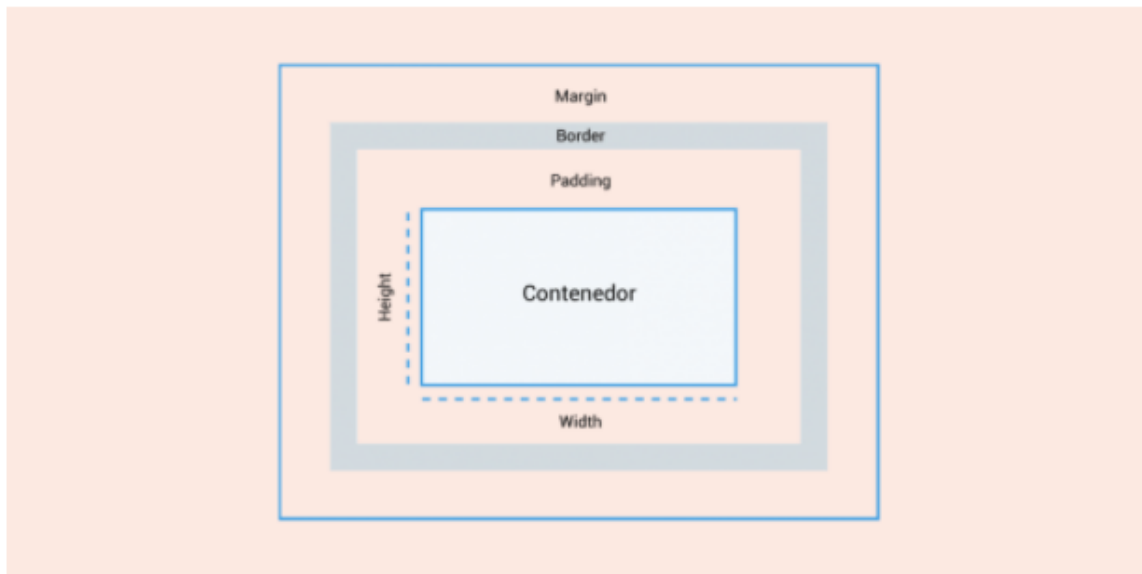
## UNIDADES DE TIEMPO			
s	Tiempo	Segundos. Usado en transiciones y animaciones.	<pre>transition: all 0.3s ease;</pre> <pre>animation-duration: 2s;</pre> Ideal para: Duración de transiciones suaves, animaciones.
ms	Tiempo	Milisegundos. 1000ms = 1s. Para tiempos muy cortos.	<pre>transition: opacity 200ms;</pre> <pre>animation-delay: 50ms;</pre> Ideal para: Micro-interacciones rápidas, delays precisos.
## UNIDADES DE ÁNGULO			
deg	Ángulo	Grados. 360deg = círculo completo. 0deg = arriba, 90deg = derecha.	<pre>transform: rotate(45deg);</pre> <pre>background: linear-gradient(90deg, red, blue);</pre> Ideal para: Rotaciones, gradientes, transformaciones.
rad	Ángulo	Radianes. 2π rad = 360deg ≈ 6.28rad.	<pre>transform: rotate(1.57rad);</pre> Ideal para: Cálculos matemáticos precisos (poco común en CSS cotidiano).
grad	Ángulo	Gradianes. 400grad = 360deg. 100grad = 90deg.	<pre>background: conic-gradient(from 100grad, red, blue);</pre> Ideal para: Gradientes cónicos (poco usado).
turn	Ángulo	Vueltas completas. 1turn = 360deg.	<pre>transform: rotate(0.5turn);</pre> <pre>animation: spin 1turn linear infinite;</pre> Ideal para: Rotaciones completas o fracciones claras (0.25turn = 90deg).

## UNIDADES DE RESOLUCIÓN			
<code>dpi</code>	Resolución	Puntos por pulgada (dots per inch).	<code>@media (min-resolution: 150dpi) { ... }</code> Ideal para: Media queries para pantallas de alta densidad.
<code>dpcm</code>	Resolución	Puntos por centímetro.	<code>@media (min-resolution: 60dpcm) { ... }</code> Ideal para: Alternativa métrica a dpi.
<code>dppx</code>	Resolución	Puntos por píxel. 1dppx = 96dpi.	<code>@media (min-resolution: 2dppx) { ... }</code> <code>image-set(url(img.png) 2x, url(img@2x.png) 2dppx);</code> Ideal para: Imágenes retina/ alta densidad.

## FUNCIONES DE CÁLCULO			
<code>calc()</code>	Función	Permite cálculos matemáticos combinando diferentes unidades.	<pre>width: calc(100% - 40px); font-size: calc(1rem + 1vw); margin-left: calc(50% - 100px);</pre> Ideal para: Combinar unidades, restar/add espacios, crear layouts híbridos.
<code>min()</code>	Función	Retorna el valor más pequeño entre los argumentos.	<pre>width: min(500px, 100%); font-size: min(3rem, 10vw);</pre> Ideal para: Limitar tamaños máximos, tipografía responsiva.
<code>max()</code>	Función	Retorna el valor más grande entre los argumentos.	<pre>width: max(300px, 50%); font-size: max(1rem, 2vw);</pre> Ideal para: Garantizar tamaños mínimos, evitar que elementos sean muy pequeños.
<code>clamp()</code>	Función	Combina min() y max(). Sintaxis: <code>clamp(mínimo, ideal, máximo)</code>	<pre>font-size: clamp(1rem, 2.5vw, 2rem); width: clamp(300px, 50%, 800px); padding: clamp(1rem, 5vw, 3rem);</pre> Ideal para: Tipografía fluida, layouts que escalan suavemente entre breakpoints. Una de las funciones más poderosas del CSS moderno.

BOX-MODEL

Modelo de cajas: márgenes, relleno y bordes

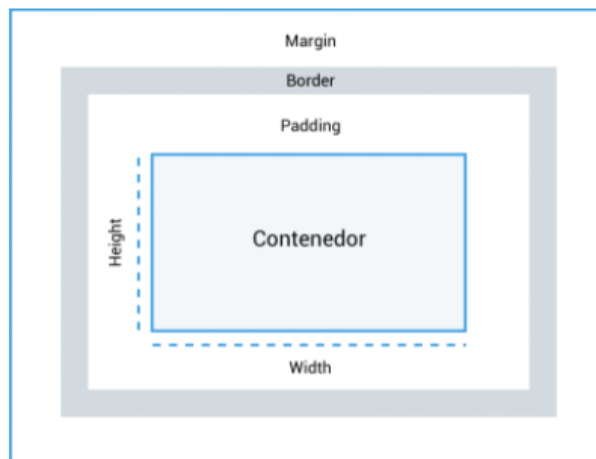


La caja tiene 4 capas (de adentro hacia afuera):

1. **Content:** El contenido real (texto, imagen).
2. **Padding:** Espacio interno transparente.
3. **Border:** Borde visible alrededor del padding.
4. **Margin:** Espacio externo transparente.

13. Modelo de cajas: márgenes, relleno y bordes

Cualquier elemento incluido en un documento HTML dispone de una estructura tipo caja que se puede modificar usando las propiedades CSS. Las propiedades más importantes de las cajas o contenedores son las siguientes:



Cálculo con `content-box` (por defecto):

- Ancho total = $200 + 20 \times 2 + 5 \times 2 = 250\text{px}$
- ¡El elemento es más ancho de lo que pedimos!

Cálculo con `border-box`:

- Ancho total = **200px** (padding y border incluidos)
- El contenido se reduce a 150px

Por qué `border-box` es mejor: Hace que el `width` sea predecible. Si dices `width: 200px`, siempre será 200px.

Cuando dos márgenes verticales se tocan, **no se suman**, sino que colapsan al valor mayor.

html

```
1 <div class="padre">
2   <div class="hijo">Contenido</div>
3 </div>
```

CSS

```
1 .padre { margin-bottom: 30px; }
2 .hijo { margin-top: 20px; }
3 /* Resultado: solo 30px de separación, no 50px */
```

Excepciones: No colapsan si:

- El elemento tiene `padding` o `border`
- El elemento es `flex` o `grid` item
- Se usa `overflow: hidden` en el padre
- Los márgenes son horizontales

13.1. Padding o relleno

Propiedad	Descripción	Valores
<code>padding-top</code> <code>padding-right</code> <code>padding-bottom</code> <code>padding-left</code>	Tamaño del relleno superior, derecho, inferior e izquierdo	longitud porcentaje
<code>padding</code>	Tamaño del relleno	longitud porcentaje {1,4}

Tabla 13.1: Padding

Ver ejemplo Padding

13.2. Margin

Propiedad	Descripción	Valores
<code>margin-top</code> <code>margin-right</code> <code>margin-bottom</code> <code>margin-left</code>	Tamaño del margen superior, derecho, inferior e izquierdo	longitud porcentaje auto
<code>margin</code>	Ancho de los márgenes	longitud porcentaje auto {1,4}

Tabla 13.2: Margin

Ver ejemplo Margin

13.3. Bordes

Propiedad	Descripción	Valores
border-top-width border-right-width border-bottom-width border-left-width	Anchura del borde superior, derecho, inferior o izquierdo	thin medium thick longitud
border-width	Anchura del borde (reducida)	thin medium thick longitud {1,4}
border-top-color border-right-color border-bottom-color border-left-color	Color del borde superior, derecho, inferior e izquierdo	color transparent
border-color	Color del borde (reducida)	color transparent {1,4}
border-top-style border-right-style border-bottom-style border-left-style	Estilo del borde superior, derecho, inferior e izquierdo	none hidden dotted dashed solid double groove ridge inset outset
border-style	Estilo del borde (reducida)	none hidden dotted dashed solid double groove ridge inset outset {1,4}
border-top border-right border-bottom border-left	Ancho, estilo y color para el borde superior, derecho, inferior e izquierdo	border-top-width border-top-style border-top-color
border	Ancho, estilo y color para los bordes (reducida)	border-width border-style border-color
border-radius	Curvatura del borde	1-4 length %

Tabla 13.3: Bordes

Ver ejemplo Bordes

Tipos de display

display es la disposición de los elementos dentro del contenedor donde se encuentran

display: block

- Ocupa todo el ancho disponible
- Acepta `width`, `height`, `margin`, `padding`
- Ejemplos: `<div>`, `<p>`, `<h1>`, `<section>`

display: inline

- Solo ocupa el espacio de su contenido
- Ignora `width`, `height`, `margin-top`, `margin-bottom`
- Ejemplos: `<span>`, `<a>`, `<strong>`

display: inline-block

- Híbrido: se comporta como inline pero acepta dimensiones
- Ideal para botones dentro de un texto

display: none

- Elimina el elemento del DOM visual (no ocupa espacio)
- Diferente de `visibility: hidden` (ocupa espacio pero no se ve)